

P3140R0

std::int_least128_t

Published Proposal, 2024-02-11

This version:

<https://eisenwave.github.io/cpp-proposals/int-least128.html>

Author:

[Jan Schultke<janschultke@gmail.com>](mailto:janschultke@gmail.com)

Audience:

SG18, LEWG, SG17, EWG, SG22

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](https://github.com/eisenwave/cpp-proposals)

Abstract

This proposal standardizes mandatory 128-bit integers types with strong library support.

Table of Contents

1	Revision history
2	Introduction
2.1	Lifting library restrictions
3	Motivation and scope
3.1	Use cases
3.1.1	Cryptography
3.1.2	Random number generation
3.1.3	Widening operations

- 3.1.3.1 64-bit modular arithmetic
- 3.1.3.2 Multi-precision operations
- 3.1.4 Fixed-point operations
- 3.1.5 High-precision time calculations
- 3.1.6 Floating-point operations
- 3.1.7 Float-to-string/String-to-float conversion
- 3.1.8 Financial systems
- 3.1.9 Universally unique identifiers
- 3.1.10 Networking
- 3.1.11 Bitsets and lookup tables
- 3.2 Utilizing hardware support
 - 3.2.1 Future-proofing for direct 128-bit support
 - 3.2.2 Support through 128-bit floating-point

4 Impact on the standard

- 4.1 C Compatibility
 - 4.1.1 `intmax_t` and `uintmax_t`
 - 4.1.2 `int_least128_t` in C
 - 4.1.3 C library impact
- 4.2 Impact on the core language
 - 4.2.1 Note on surprising semantics
- 4.3 Impact on the library
 - 4.3.1 Language support library
 - 4.3.2 Metaprogramming library
 - 4.3.3 General utilities library
 - 4.3.4 Containers library
 - 4.3.5 Iterators library
 - 4.3.6 Ranges library
 - 4.3.7 Algorithms library
 - 4.3.8 Numerics library
 - 4.3.9 Time library
 - 4.3.10 Localization library
 - 4.3.11 Input/output library
 - 4.3.12 Concurrency support library

5 Impact on implementations

5.1 Estimated implementation effort

5.2 `std::ranges::iota_view` ABI issue

5.2.1 Affected implementations

5.2.2 Cause of ABI break

5.2.3 Possible solution

5.2.4 What about `iota_view<std::int_least128_t>`?

6 Impact on existing code

6.1 Possible semantic changes

6.2 Impact on overload sets

6.2.1 Proposed solution

6.3 Possible assumption violations

6.4 `std::bitset` constructor semantic changes

6.4.1 Proposed solution

7 Design considerations

7.1 Why no standard integer type?

7.2 Why no mandatory `std::int128_t` type?

7.3 Why no `std::int_least256_t` type?

7.4 Why no class type?

7.5 Why no bit-precise integers?

7.5.1 `_BitInt` has no existing C++ work

7.5.2 `_BitInt` has less motivation in C++

7.5.3 `_BitInt` should be exposed as a class type

7.5.4 `_BitInt` is not a replacement for standard integers

7.5.5 `_BitInt` does not guarantee 128-bit support

7.5.6 `_BitInt` requires more library effort

7.5.7 `_BitInt` breaks more existing code

7.5.8 `_BitInt` has teachability issues

7.5.9 `_BitInt` in C may be too permissive

7.5.10 `_BitInt` false dichotomy

7.6 Why not make it optional?

7.6.1 Implementation effort is not too high

- 7.6.2 Software emulation is acceptable
- 7.7 Should extended integer semantics be changed?
- 7.8 Do we need new user-defined literals?
- 7.9 Why no `std::div`?
- 7.10 What about the `std::ratio` dilemma?

8 Implementation experience

- 8.1 Existing 128-bit integer types
 - 8.1.1 `__int128` (GNU-like)
 - 8.1.2 `__int128` (CUDA)
 - 8.1.3 `std::_Signed128`, `std::_Unsigned128`
 - 8.1.4 `_BitInt(128)`
- 8.2 Library implementation experience
 - 8.2.1 `std::to_integer`
 - 8.2.2 `<type_traits>`
 - 8.2.3 `std::cmp_xxx`
 - 8.2.4 `<bitset>`
 - 8.2.5 `<charconv>`
 - 8.2.6 `<format>`
 - 8.2.7 `<bit>`
 - 8.2.8 `std::to_string`
 - 8.2.9 `std::gcd`, `std::lcm`
 - 8.2.10 `<random>`
 - 8.2.11 `std::midpoint`
 - 8.2.12 `std::xxx_sat`
 - 8.2.13 `std::abs`
 - 8.2.14 `<cmath>`
 - 8.2.15 `std::valarray`
 - 8.2.16 `<linalg>`
 - 8.2.17 `<cstdio>`
 - 8.2.18 `std::atomic`

9 Proposed wording

- 9.1 Header `<version>`

- 9.2 Header `<cstdint>`
- 9.3 Header `<inttypes.h>`
- 9.4 Class template `bitset`
- 9.5 Numeric conversions
- 9.6 Iota view
- 9.7 Absolute values
- 9.8 Header `<cinttypes>`
- 9.9 Atomic operations

10 Acknowledgements

References

Normative References

Informative References

§ 1. Revision history

This is the first revision.

§ 2. Introduction

128-bit integers have numerous practical uses, and all major implementations (MSVC, GCC, LLVM) provide 128-bit integers already. Among C++ users, there has been great interest in standardizing integers beyond 64 bits for a long time. With the new wording in the C23 standard for `intmax_t` (see [§ 4.1 C Compatibility](#)), one of the last obstacles has been removed.

The goal of this paper is to obtain a mandatory ≥ 128 -bit integer type with no core language changes and strong support from the C++ standard library. To accomplish this, the mandatory aliases `std::int_least128_t` and `std::uint_least128_t` are proposed. Note that any non-malicious implementation would be required to define `std::uint128_t` if possible, so standardizing the minimum-width types is standardizing exact-width types by proxy.

While the definition of these aliases is trivial, mandating them also implies library support from `<format>`, `<bit>`, `<cmath>`, `<limits>`, and other facilities. After extensive investigation, it was determined that the [§ 4 Impact on the standard](#) and [§ 5 Impact on implementations](#) is relatively low.

§ 2.1. Lifting library restrictions

The standard library contains a large amount of artificial hurdles which make it impossible to provide library support for extended integers. The current standard already permits the implementation to provide additional extended (fundamental) integer types in addition to the standard integers (`int`, `long`, etc.). However, even if there exists an extended 128-bit integer, among other issues:

- `std::to_string(std::int128_t)` cannot exist,
- `std::bitset` cannot be constructed from it (without truncating to `unsigned long long`), and
- `std::abs(std::int128_t)` cannot exist.

It would not be legal for an implementation to provide such additional overloads because it would change the meaning of well-formed programs.

EXAMPLE 1

The following code is a well-defined C++20 program which uses the optional `std::int128_t` type.

```
#include <string>
#include <concepts>

struct S {
    template <typename T>
        requires std::same_as<T, long long> || std::same_as<T, std::int128_t>
        operator T() const { return 0; }
};

int main() {
    std::to_string(S{});
}
```

This code must always call `std::to_string(long long)`. If `std::int128_t` was not the same type as `long long` and the implementation added an overload `std::to_string(std::int128_t)` in spite of the standard, the call to `std::to_string` would be ambiguous.

The implementation has *some* ability to add overloads, stated in [global.functions] paragraph 2:

A call to a non-member function signature described in [support] through [thread] and [depr] shall behave as if the implementation declared no additional non-member function signatures.

This condition is not satisfied. If a `std::to_string(std::int128_t)` overload existed, the behavior would not be *as if* the implementation declared no additional signature.

NOTE: `std::uint128_t` is not a compiler extension; it's an optional feature. C23 [N3047] subclause 7.22.1.1 [Exact-width integer types] paragraph 3 requires implementations to "define the corresponding typedef names" if there exists a padding-free integer type with 128 bits.

Even if you don't find this example convincing, at best, `std::to_string` and other library support would be optional. There are also functions which undeniably cannot exist, like `std::bitset::to_u128` (there are only `to_ulong` and `to_ullong`). It would be highly undesirable to have a 128-bit type but no standard library support which is documented in the standard, and which is optional on a per-function basis, with no feature-testing macros. Wording changes should be made to clean up this environment.

§ 3. Motivation and scope

There are compelling reasons for standardizing a 128-bit integer type:

1. **Utility:** 128-bit integers are extremely useful in a variety of domains.
2. **Uniformity:** Standardization would unify the many uses under a common name and ideally, common ABI.
3. **Existing practice:** 128-bit integers are already implemented in multiple compilers (see [§ 8.1 Existing 128-bit integer types](#)).
4. **Performance:** It is difficult, if not impossible to optimize 128-bit operations in software as well as the compiler could do for a builtin type (see [§ 3.2 Utilizing hardware support](#)).
5. **Low impact:** The [§ 4 Impact on the standard](#) and [§ 5 Impact on implementations](#) is reasonable.

§ 3.1. Use cases

A GitHub code search for `/int128|int_128/ language:c++` yields 150K files, and a language-agnostic search for `/int128|int_128/` yields more than a million.

While it is impossible to discuss every one of these, I will introduce a few use cases of 128-bit integers.

§ 3.1.1. Cryptography

128-bit integers are commonly used in many cryptographic algorithms:

- Most notably, AES-128 uses a 128-bit key size. AES variants with wider key sizes still use a block size of 128 bits.
- Various other block ciphers such as Twofish and Serpent also have key and/or block sizes of 128 bits.
- MD5 hashes produce 128-bit output.
- SHA-2 and SHA-3 produce outputs beyond 128-bit, but outputs can be truncated to 128-bit, or represented as a pair/array of 128-bit integers.

For example, the [AddRoundKey step in AES](#) is simply a 128-bit bitwise XOR.

To be fair, the utility of 128-bit integers in cryptographic applications is often limited to providing storage for blocks and keys.

§ 3.1.2. Random number generation

Some random number generators produce 128-bit numbers.

For example, the [CSRPNG](#) (cryptographically secure pseudo-random number generator) [Fortuna](#) uses a block cipher to produce random numbers. When a 128-bit block cipher is used, the output is naturally 128-bit as well. Fortuna is used in the implementation of `/dev/random` in FreeBSD 11, and in AppleOSes since 2020.

Some PRNGs use a 128-bit state, such as [xorshift128](#).

EXAMPLE 2

The following code is based on [\[Marsaglia\]](#), with some changes.

```
std::uint32_t xor128(std::uint32_t x[4]) {  
    std::uint32_t t = x[3];  
    t ^= t << 11;  
    t ^= t >> 8;  
  
    x[3] = x[2]; x[2] = x[1]; x[1] = x[0];  
  
    x[0] ^= t ^ (x[0] >> 19);  
    return x[0];  
}
```

This can be expressed more elegantly using 128-bit integers:

```
std::uint32_t xor128(std::uint128_t& x) {  
    std::uint32_t t = x >> 96;  
    t ^= t << 11;  
    t ^= t >> 8;  
  
    x = (x << 32) | (t ^ (std::uint32_t(x) ^ (std::uint32_t(x) >> 19)));  
    return x;  
}
```

Generally speaking, there is a large amount of code that effectively performs 128-bit operations, but operates on sequences of 32-bit or 64-bit integers. In the above example, it is not immediately obvious that the `x[3] = x[2]; ...` line is effectively performing a 32-bit shift, whereas `x << 32` is self-documenting.

[\[P2075R3\]](#) proposes counter-based Philox engines for the C++ standard library, and has been received positively. The [\[DEShawResearch\]](#) reference implementation makes use of 128-bit integers.

§ 3.1.3. Widening operations

128-bit arithmetic can produce optimal code for mixed 64/128-bit operations, for which there is already widespread hardware support. Among other instructions, this hardware support includes:

Operation	x86_64	ARM	RISC-V
64-to-128-bit unsigned multiply	<code>mul</code> : output to register pair <code>rdx:rax</code>	<code>umulh</code> for high bits, <code>mul</code> for low bits	<code>mulhu</code> for high bits, <code>mulu</code> for low bits
64-to-128-bit signed multiply	<code>imul</code> : output to register pair <code>rdx:rax</code>	<code>smulh</code> for high bits, <code>mul</code> for low bits	<code>mulsu</code> for high bits, <code>muls</code> for low bits
128-to-64-bit unsigned divide	<code>div</code> : <code>rax</code> = quotient, <code>rdx</code> = remainder	✗	<code>divu</code> (RV128I)
128-to-64-bit signed divide	<code>idiv</code> : <code>rax</code> = quotient, <code>rdx</code> = remainder	✗	<code>divs</code> (RV128I)
64-to-128-bit carry-less multiply	<code>pclmulqdq</code> : output 128 bits to <code>xmm</code> register	<code>pmull</code> for low bits, <code>pmull2</code> for high bits	<code>clmul</code> for low bits, <code>clmulh</code> for high bits

Some operating systems also provide 64/128-bit operations. For example, the Windows API provides a `Multiply128` function.

A more general solution was proposed by [P3018R0], which supports widening multiplication through a `std::mul_wide` function, which yields the low and high part of the multiplication as a pair of integers. Such utilities would be useful in generic code where integers of any width can be used. For 64-to-128-bit, it's obviously more ergonomic to cast operands to `std::int128_t` prior to an operation.

EXAMPLE 3

The [Stockfish] chess engine has a `mul_hi64` function which yields the high part of a 64-bit multiplication:

```
inline uint64_t mul_hi64(uint64_t a, uint64_t b) {
    #if defined(__GNUC__) && defined(IS_64BIT)
        __extension__ using uint128 = unsigned __int128;
        return (uint128(a) * uint128(b)) >> 64;
    #else
        // ...
    }
```

§ 3.1.3.1. 64-bit modular arithmetic

To perform modular arithmetic with a 64-bit modulo, 128-bit integers are needed. For example, when computing $(a * x) \% m$ between 64-bit unsigned integers `a`, `x`, and `m`, the multiplication between `a` and `x` is already mod 2^{64} , and the result would be incorrect unless `m` was a power of two.

128-bit operations are used in implementations of `std::linear_congruential_engine` (see § 8.2.10 <random> for implementation experience). Linear congruential engines use modular arithmetic, and since the user can choose a modulo arbitrarily, the issue is unavoidable.

NOTE: A popular workaround for linear congruential generators is to choose the modulo to be 2^{64} or 2^{32} . This means that division is not required at all.

§ 3.1.3.2. Multi-precision operations

For various applications (cryptography, numerics, etc.) arithmetic with large widths is required. For example, the RSA (Rivest–Shamir–Adleman) cryptosystem typically uses key sizes of 2048 or 4096. "Scripting languages" also commonly use an infinite-precision integer type. For example, the `int` type in Python has no size limit.

Multi-precision operations are implemented through multiple widening operations. For example, to implement N-bit multiplication, the number can be split into a sequence of 64-bit "limbs", and long multiplication is performed. Since this involves a carry between digits, a 64-to-128-bit widening operation is required.

[BoostMultiPrecision] uses a 128-bit integer as a `double_limb_type`. This type is used extensively in the implementation of multi-precision arithmetic.

NOTE: The introduction of bit-precise integers (§ 7.5 Why no bit-precise integers?) does not obsolete multi-precision libraries because infinite-precision numbers like Python's `int` cannot be implemented using a constant size.

§ 3.1.4. Fixed-point operations

While 64-bit integers are sufficient for many calculations, the amount of available bits is reduced when the 64 bits are divided into an integral and fractional part. This may cause issues in § 3.1.8 Financial systems.

Furthermore, fixed-point arithmetic with a double-wide operand can emulate integer division, which is a relatively expensive operation, even with hardware support.

EXAMPLE 4

128-bit integers allow us to implement a division by three without integer division:

```
std::uint64_t div3(std::uint64_t x) {  
    // 1 / 3 as a Q63.65 fixed-point number:  
    constexpr std::uint_fast128_t reciprocal_3 = 0xAAAA'AAAA'AAAA'AAAB;  
    return (x * reciprocal_3) >> 65; // equivalent to return x / 3;  
}
```

While modern compilers perform this [strength reduction optimization](#) for constant divisors already, they don't perform it for frequently reused non-constant divisors.

For such divisors, it can make sense to pre-compute the reciprocal and shift constants and use them many times for faster division. Among other libraries, [libdivide](#) uses this technique (using a pair of 64-bit integers, which effectively forms a 128-bit integer).

NOTE: 3 is a "lucky" case because all nonzero bits fit into a 64-bit integer. The amount of digits required differs between divisors.

§ 3.1.5. High-precision time calculations

64-bit integers are somewhat insufficient for high-precision clocks, if large time spans should also be covered. When counting nanoseconds, a maximum value of $2^{63}-1$ can only represent approximately 9 billion seconds, or 7020 years. This is enough to keep time for the foreseeable future, but is insufficient for representing historical data long in the past.

This makes 64-bit integers insufficient for some time calculations, where 128-bit integers would suffice. Alternatively, 64-bit floating-point numbers can provide a reasonable trade-off between resolution and range.

`timespec` is effectively a 128-bit type in POSIX (`std::timespec` in C++), since both the seconds and nanoseconds part of the class are 64-bit integers (assuming that `std::time_t` is 64-bit).

[\[Bloomberg\]](#) uses 128-bit integers to safeguard against potential overflow in time calculations (see [bsls_timeutil.cpp](#))

§ 3.1.6. Floating-point operations

The implementation of IEEE 754/IEC-559 floating-point operations often involves examining the bit-representation of the floating-point number through an unsigned integer.

The C++ standard provides `std::float128_t`, but no matching 128-bit integer type, which makes this more difficult.

EXAMPLE 5

Using 128-bit integers, `std::signbit(std::float128_t)` can be implemented as follows:

```
constexpr bool signbit(float128_t x) {  
    return bit_cast<uint128_t>(x) >> 127;  
}
```

EXAMPLE 6

Using 128-bit integers, `std::isinf(std::float128_t)` can be implemented as follows:

```
constexpr float128_t abs(float128_t x) {  
    return bit_cast<float128_t>(bit_cast<uint128_t>(x) & (uint128_t)(-1))  
}  
  
constexpr bool isinf(float128_t x) {  
    return bit_cast<uint128_t>(abs(x)) == 0x7fff'0000'0000'0000'0000'0000  
}
```

NOTE: Infinity for *binary128* numbers is represented as any sign bit, 15 exponent bits all set to 1, and 112 mantissa bits all set to 0.

[Bloomberg] uses 128-bit integers as part of a 128-bit decimal floating point implementation, among other uses. Decimal floating-point numbers are commonly used in financial applications and are standard C23 types (e.g. `_Decimal128`) since [N2341]. In C++, a software implementation is necessary.

§ 3.1.7. Float-to-string/String-to-float conversion

The [\[Dragonbox\]](#) binary-to-decimal conversion algorithm requires an integer type that is twice the width of the converted floating-point number. To convert a floating-point number in [binary64](#) format, a 128-bit integer type is used.

Similarly, [\[fast_float\]](#) uses 128-bit numbers as part of decimal-to-binary conversions. This library provides an efficient [from_chars](#) implementation.

§ 3.1.8. Financial systems

128-bit integers can be used to represent huge monetary values with high accuracy. When representing cents of a dollar as a 64-bit integer, a monetary value of up to 184.5 quadrillion dollars can be represented. However, this value shrinks dramatically when using smaller fractions.

Since 2005, stock markets are legally required to accept price increments of \$0.0001 when the price of a stock is $\leq \$1$ (see [\[SEC\]](#)). At this precision, 1.84 quadrillion dollars can be represented. Using a uniform precision of ten thousandths would prove problematic when applied to other currencies such as Yen, which forces the complexity of variable precision on the developer.

Even more extremely, the smallest fraction of a Bitcoin is a Satoshi, which is a hundred millionth of a Bitcoin. 2^{63} Satoshis equal approximately 92 billion BTC. In 2009, a Bitcoin was worth less than a penny, so a monetary value of only 920 million USD could be represented in Satoshis.

In conclusion, simply storing the smallest relevant fraction as a 64-bit integer is often insufficient, especially when this fraction is very small and exponential price changes are involved. Rounding is not always an acceptable solution in financial applications.

[\[NVIDIA\]](#) mentions fixed-point accounting calculations as a possible use case of the `__int128` type, which is a preview feature of NVIDIA CUDA 11.5.

[\[TigerBeetle\]](#) discusses why 64-bit integers have been retired in favor of 128-bit integers to store financial amounts and balances in the TigerBeetle financial accounting database. The aforementioned sub-penny requirement is part of the motivation.

§ 3.1.9. Universally unique identifiers

A 128-bit integer can be used to represent a [UUID](#) (Universally Unique Identifier). While 64-bit integers are often sufficient as a unique identifier, it is quite likely that two identical identifiers are

chosen by a random number generator over a long period of time, especially considering the [Birthday Problem](#). Therefore, at least 128 bits are typically used for such applications.

EXAMPLE 7

The following code generates a UUIDv4, represented as an unsigned 128-bit integer.

```
std::uint128_t random_uuid_v4() {
    return std::experimental::randint<std::uint128_t>(0, -1)
        & 0xffff'ffff'ffff'003f'ff0f'ffff'ffff'ffff // clear version a
        | 0x0000'0000'0000'0080'0040'0000'0000'0000; // set to version
}
```

The [ClickHouse](#) database management system defines their `UUID` type through `StrongTypedef<UInt128, struct UUIDTag>`.

§ 3.1.10. Networking

IPv6 addresses can be represented as a 128-bit integer. This may be a convenient representation because bitwise operations for masking and accessing individual bits or bit groups may be used. Implementing these is much easier using a 128-bit integers compared to multi-precision operations using two 64-bit integers.

EXAMPLE 8

An IPv6 address in link-local address format can be identified as follows:

```
std::uint128_t ipv6 = /* ... */;
constexpr auto mask10 = 0x3ff;
if ((ipv6 & mask10) != 0b1111111010) /* wrong prefix */;

constexpr auto mask54 = (std::uint64_t(1) << 54) - 1;
if ((ipv6 >> 10 & mask54) != 0) /* expected 54 zeros */;

constexpr auto mask64 = std::uint64_t(-1);
interface_identifier = (data >> 64) & mask64;
```

The [ClickHouse](#) database management system defines their `IPv6` type through `StrongTypedef<UInt128, struct IPv6Tag>`.

§ 3.1.11. Bitsets and lookup tables

A popular technique for optimizing small lookup tables in high-performance applications is to turn them into a bitset. 128 bits offer additional space over 64 bits.

EXAMPLE 9

Axis vectors $((-1, 0, 0), (1, 0, 0), \dots, \text{or } (0, 0, 1))$ can be represented as integers in range $[0, 6)$. This requires three bits of storage, and a lookup table for the cross product of two vectors requires $3^3 = 81$ bits. The cross product can be computed as follows:

```
unsigned cross(unsigned a, unsigned b) {
    [[assume(a < 6 && b < 6)]];
    constexpr std::uint128_t lookup = 0x201'6812'1320'8941'06c4'ec21'a94
    return (lookup >> (a * 6 + b * 3)) & 0b111;
}
```

This is significantly faster than computing a cross product between triples of `int` or `float` using multiplication and subtraction.

Using unsigned integers as lookup tables is a very popular technique in chess engines, and commonly referred to as [Bitboard](#). The [\[px0\]](#) chess engine uses a 90-bit board, stored in a 128-bit integer.

NOTE: Compilers can only perform an "array-to-bitset optimization" to a limited extent at this time. Clang is the only compiler which performs it, and only for arrays of `bool`.

NOTE: `std::bitset` does not offer a way to extract ranges of bits, only individual bits. Therefore, it would not have been of much help in the example. Furthermore, `std::bitset` has runtime checks and potentially throws exceptions, which makes it unattractive to some language users.

§ 3.2. Utilizing hardware support

§ 3.2.1. Future-proofing for direct 128-bit support

Hardware support for 64/128-bit mixed operations is already common in x86_64 and ARM. It is also conceivable that hardware support for 128-bit integer arithmetic will be expanded in the foreseeable

future. The RISC-V instruction set architecture has a 128-bit variant named RV128I, described in [\[RISC-V\]](#), although no implementation of it exists yet.

When hardware support for 128-bit operations is available, but the source code emulates these in software, the burden of fusing multiple 64-bit operations into a single 128-bit operation is put on the optimizer.

For example, multiple 64-bit multiplications may be fused into a single 64-to-128-bit multiplication. x86_64 already provides hardware support in this case (see [§ 3.1.3 Widening operations](#)), however, the language provides no way of expressing such an operation through integer types.

§ 3.2.2. Support through 128-bit floating-point

On hardware which provides native support for `std::float128_t` (see [\[Wikipedia\]](#) for a list), integer division up to 113 bits can be implemented in terms of floating-point division, and this is possibly the fastest routine. For such instruction selection, the 113-bit division must be recognized by the compiler. It is very unlikely that the hundreds of operations comprising a software integer division could be recognized as such.

128 bits is obviously more than 113 bits, so not every operation can be performed this way. However, modern optimizing compilers keep track of range constraints of values.

EXAMPLE 10



The optimizer may make the following decisions when performing 128-bit integer division:

1. If the divisor is zero, mark the operation undefined behavior for the purpose of compiler optimization, or emit `ud2`.
2. Otherwise, if the divisor and dividend are constant, compute the result.
3. Otherwise, if the divisor is constant and greater than the dividend, yield zero.
4. Otherwise, if the divisor is constant, perform strength reduction ([§ 3.1.4 Fixed-point operations](#)), making the division a shift and multiplication.
5. Otherwise, if the divisor is a power of two, count the trailing zeros and perform a right-shift.
6. Otherwise, if both operands are $2^{64}-1$ or less, perform 64-bit integer division.
7. Otherwise, if one of the operands is $2^{64}-1$ or less, perform 128-to-64-bit ([§ 3.1.3 Widening operations](#)) division.
8. **Otherwise, if both operands are $2^{113}-1$ or less, and if there is hardware support for 128-bit floating point numbers, perform floating-point division.**
9. Otherwise, use a software implementation of 128-bit division.

ISO C++ does not offer a mechanism through which implementations can be chosen based on optimizer knowledge. What is easy for the implementation is difficult for the user, which makes it very compelling to provide a built-in type.

NOTE: The pre-computation in bullet 4 must not be done in C++ because the cost of computing the reciprocal is as high as the division itself. The user must be guaranteed that the entire pre-computation of shift and factor is constant-folded, and this is generally impossible because optimization passes are finite.

NOTE: Historically, floating-point division in hardware was used to implement integer division. The x87 circuitry for dividing 80-bit floating-point numbers could be repurposed for 64-bit integer division. This strategy is still many times faster than software division. Intel desktop processors have received dedicated integer dividers starting with Cannon Lake.

§ 4. Impact on the standard

First and foremost, this proposal mandates the following integer types in `<cstdint>`:

```
using int_least128_t = /* signed integer type */;
using uint_least128_t = /* unsigned integer type */;

using int_fast128_t = /* signed integer type */;
using uint_fast128_t = /* unsigned integer type */;

using int128_t = /* signed integer type */; // optional
using uint128_t = /* unsigned integer type */; // optional

// TODO: define corresponding macros/specializations in <inttypes>, <climits>
```

This change in itself is almost no change at all. The implementation can already provide `int_least128_t` while complying with the C++11 standard. Challenges only arise when considering the impact of these new types on the rest of the standard library, and possibly C compatibility.

NOTE: A compliant libstdc++ implementation could define all aliases as `__int128`.

§ 4.1. C Compatibility

This proposal makes the assumption that C++26 will be based on C23. Any attempt of standardizing 128-bit integers must also keep possible compatibility with the C standard in mind.

§ 4.1.1. `intmax_t` and `uintmax_t`

In particular, `intmax_t` has historically prevented implementations from providing integer types wider than `long long` without breaking ABI compatibility. A wider integer type would change the width of `intmax_t`.

C23 has relaxed the definition of `intmax_t`. [N3047], 7.22.1.5 [Greatest-width integer types] currently defines `intmax_t` as follows:

The following type designates a signed integer type, other than a bit-precise integer type, capable of representing any value of any signed integer type with the possible exceptions of signed bit-precise integer types and of signed extended integer types that are wider than `long long` and that are referred by the type definition for an exact width integer type:

```
intmax_t
```

For `intmax_t` to not be `int_least128_t`, there must exist an `int128_t` alias for the same type. GCC already provides an `__int128` type which satisfies the padding-free requirement and could be exposed as `int128_t`.

In conclusion, it is possible to provide a `std::int_least128_t` alias with equivalent semantics in C and C++, and with no ABI break.

§ 4.1.2. `int_least128_t` in C

`std::int_least128_t` does not force `int_least128_t` to exist in C. In principle, C++ compilers can disable support for the type in C mode, so that there is effectively no impact.

However, this would be somewhat undesirable because there would be no mandatory interoperable type. C users would use `_BitInt(128)` or `__int128` and C++ users would use `std::int128_t`, which would only be `__int128` by coincidence. For the sake of QoI, implementations should expose the corresponding alias in C as well (which they are allowed to).

To make the type mandatory in both languages, cooperation from WG14 is needed.

§ 4.1.3. C library impact

A C++ implementation is required to provide compatibility headers as per [support.c.headers] which have equivalent semantics to the headers in C, with some details altered. The affected candidates are those listed in Table 40: C headers [tab:c.headers].

Header	Impact of extended integers
<code><inttypes.h></code>	Define macro constants
<code><limits.h></code>	Define macro constants
<code><stdatomic.h></code>	Define type aliases
<code><stdint.h></code>	Define type aliases
<code><stdio.h></code>	Support 128-bit <code>printf/scanf</code> optionally

There is no impact on other C headers. Most of the issues are trivial and have no runtime C library impact. The only thing worth noting is that 128-bit support from `printf/scanf` would be required (§ 8.2.17 `<cstdio>` for implementation experience).

NOTE: This support is made optional, so that C++ implementations are able to keep using the system's C library. Otherwise, the C++ implementation could only guarantee 128-bit `printf` support if it was part of the C++ runtime library.

NOTE: If additionally, a C implementation wanted to support `int_least128_t`, it would need to add extended integer support in a few other places. For example, `stdbit.h` requires type-generic functions to support all extended integers.

§ 4.2. Impact on the core language

The proposal makes no changes to the core language because existing semantics of extended integers are sufficient (see § 7.7 [Should extended integer semantics be changed?](#) for discussion). See also § 7.8 [Do we need new user-defined literals?](#).

§ 4.2.1. Note on surprising semantics

It is worth noting that the existence of `std::uint_least128_t` leads to some oddities:

- The result of the `sizeof` operator can be a 128-bit integer.
- The underlying type of enumerations can be a 128-bit integer.
- Previously ill-formed integer literals could now be of 128-bit integer type.
- The conditions in `#if` preprocessing directives are evaluated as if operands had the same representation as `intmax_t` or `uintmax_t`, which means that 128-bit integers cannot be used in this context, or the values would be truncated.

However, none of this is a new issue introduced by this proposal. Any compliant implementation could already have produced this behavior, assuming it supported 128-bit integers as an optional extended integer type.

NOTE: The fact that the preprocessor doesn't operate on the widest integer type, but on `intmax_t` will need to be addressed. However, this is a general problem with rebasing on C23 and not within the scope of this proposal.

§ 4.3. Impact on the library

Find below a summary of issues that arise from the introduction of 128-bit integers in the C++ standard library. One common issue is that aliases such as `size_type` and `difference_type` within containers, iterators, and other types can be a 128-bit integer. The same applies to `std::size_t`, `std::ptrdiff_t`, `std::intmax_t`.

The proposal does not force library maintainers to re-define any of these aliases; it's just a possibility. Whether to define them as such is a QoI issue in general, and won't be discussed further.

§ 4.3.1. Language support library

Issue: `std::div` may need an overload for 128-bit integers.

Action: ✓ None because we don't support it (see § 7.9 [Why no std::div?](#)).

Issue: `std::to_integer` may need 128-bit support.

Action: ✓ None (see § 8.2.1 [std::to_integer](#) for implementation experience).

Issue: `<version>` needs 128-bit integer feature-testing macro.

Action: ⚠ Add macros (see § 9.1 [Header <version>](#) for wording).

Issue: `<numeric_limits>` needs a trait for 128-bit integers.

Action: ✓ None.

Issue: `<climits>` needs additional constants for 128-bit integers.

Action: ✓ None.

Issue: `<cstdint>` needs to explicitly require support for 128-bit integers in its synopsis.

Action: ⚠ Define aliases (see § 9.2 [Header <cstdint>](#) for wording).

Issue: `<inttypes.h>` needs to support 128-bit only optionally.

Action: ⚠ Make support optional (see § 9.3 [Header <inttypes.h>](#) for wording).

§ 4.3.2. Metaprogramming library

Issue: `std::is_integral` needs to support 128-bit integers.

Action: ✓ None (see § 8.2.2 [<type_traits>](#) for implementation experience).

Issue: `std::make_signed` and `std::make_unsigned` require 128-bit support.

Action: ✓ None (see § 8.2.2 [<type_traits>](#) for implementation experience).

Issue: `std::ratio` currently accepts non-type template arguments of `std::intmax_t`. `std::intmax_t` is no longer the widest integer type and changing the type of NTTP to `std::uint_least128_t` would be an ABI break because the type of template argument participates in name mangling.

Action: ✓ None (see [§ 7.10 What about the std::ratio dilemma?](#) for discussion).

§ 4.3.3. General utilities library

Issue: Integer comparison functions (`std::cmp_equal` et al.) require 128-bit support.

Action: ✓ None (see [§ 8.2.3 std::cmp_xxx](#) for implementation experience).

Issue: `std::integer_sequence` needs to support 128-bit integers.

Action: ✓ None.

Issue: `std::bitset` could receive an additional constructor taking `std::uint_least128_t`.

Action: ⚠ Add such a constructor (see [§ 9.4 Class template bitset](#) for wording [§ 6.4 std::bitset constructor semantic changes](#) for discussion).

Issue: `std::bitset` could receive an additional `to_u128` function, similar to `to_ullong`.

Action: ⚠ Add such a function (see [§ 9.4 Class template bitset](#) for wording and [§ 8.2.4 <bitset>](#) for implementation experience).

Issue: `std::to_chars` and `std::from_chars` need to support 128-bit integers.

Action: ✓ None (see [§ 8.2.5 <charconv>](#) for implementation experience).

Issue: `<format>` needs to support 128-bit integers.

Action: ✓ None (see [§ 8.2.6 <format>](#) for implementation experience).

Issue: `basic_format_parse_context::check_dynamic_spec` might need 128-bit integers support.

Action: ✓ None, it doesn't (see [§ 8.2.6 <format>](#) for implementation experience).

Issue: `basic_format_arg` might need support for 128-bit integers.

Action: ✓ None, it doesn't (see [§ 8.2.6 <format>](#) for implementation experience).

Issue: `<bit>` needs to support 128-bit integers.

Action: ✓ None (see [§ 8.2.7 <bit>](#) for implementation experience).

Issue: `std::to_string` could support 128-bit types.

Action: ⚠ Add overloads (see [§ 9.5 Numeric conversions](#) for wording and [§ 8.2.8 std::to_string](#) for implementation experience).

§ 4.3.4. Containers library

Issue: The extents and index types of `std::mdspan` could be 128-bit integers. This is also the case for type aliases of `std::strided_slice`. The exposition-only helper `integral-constant-like` now also includes 128-bit integers.

Action: ✓ None. All these issues are either QoI or don't impact existing implementations substantially.

§ 4.3.5. Iterators library

Issue: The exposition-only helper `integral-constant-like` now also includes 128-bit integers. Generally, 128-bit integers would be a valid `difference_type` and an implementation needs to consider this when defining concepts that use integers in any way.

Action: ✓ None.

NOTE: As long as `std::is_integral` (and by proxy, `std::integral`) is correct, the existing wording should be unaffected.

§ 4.3.6. Ranges library

Issue: `std::ranges::iota_view<long long>::iterator::difference_type` is required to be a 128-bit integer if `long long` is not 128-bit, and such a type exists. This is not the case in libc++ and MSVC STL at this time, where `difference_type` is `long long` and `class std::_Signed128` respectively.

Action: ⚠ Relax wording to prevent breaking ABI (see § 9.6 [Iota view](#) for wording and § 5.2 [std::ranges::iota_view ABI issue](#) for discussion).

Issue: `std::cartesian_product_view::size` may now return a 128-bit integer. The standard recommends to use a type which is sufficiently wide to store the product of sizes of underlying ranges. A similar issue arises for `std::cartesian_product_view::iterator`.

Action: ✓ None.

NOTE: The choice of integer type used to be (and still is) implementation-defined.

§ 4.3.7. Algorithms library

Issue: `std::gcd`, `std::lcm`, and `std::midpoint` need to support 128-bit integers.

Action: ✓ None (see § 8.2.9 `std::gcd`, `std::lcm` for implementation experience).

Issue: Saturating arithmetic functions and `saturate_cast` need to support 128-bit integers.

Action: ✓ None (see § 8.2.12 `std::xxx_sat` for implementation experience).

§ 4.3.8. Numerics library

Issue: Various random number generators and `std::uniform_int_distribution` need to support 128-bit types.

Action: ✓ None (see § 8.2.10 `<random>` for implementation experience).

Issue: `std::seed_seq` needs to support `std::initializer_list<std::uint128_t>`.

Action: ✓ None (see § 8.2.10 `<random>` for implementation experience).

Issue: `std::valarray` needs to support 128-bit integers.

Action: ✓ None (see § 8.2.15 `std::valarray` for implementation experience)

Issue: For most `<cmath>` functions, an additional overload taking 128-bit integers would need to be defined.

Action: ✓ None (see § 8.2.14 `<cmath>` for implementation experience.)

Issue: `std::abs` could receive an additional 128-bit overload.

Action: ⚠ Add an overload (see § 9.7 *Absolute values* for wording and § 8.2.13 `std::abs` for implementation experience).

Issue: The `<linalg>` library needs 128-bit support.

Action: ✓ None (see § 8.2.16 `<linalg>` for implementation experience).

§ 4.3.9. Time library

Issue: Significant portions of `<chrono>` use `std::ratio`, which has `std::intmax_t` template parameters.

Action: ✓ None (see § 7.10 *What about the `std::ratio` dilemma?* for discussion).

§ 4.3.10. Localization library

Issue: `std::num_get` and `std::num_put` could use `std::uint_least128_t` overloads for `do_get`.

Action: ✓ None.

NOTE: This would be a an ABI break if changes were made. `std::num_get` relies on virtual member functions, and modifying the vtable breaks ABI. `std::printf` and `std::scanf` can be used for locale-dependent formatting without breaking ABI. `std::format` and `std::print` provide locale-independent alternatives.

§ 4.3.11. Input/output library

Issue: `std::num_get` and `std::num_put` don't support 128-bit integers. By proxy, extraction with `std::ostream::operator<<` and insertion with `std::istream::operator>>` would not work.

Action: ✓ None.

NOTE: The standard doesn't require these to work for all integer types, only for standard integer types. Any change would be an ABI break, so these facilities could be left untouched. Unfortunately, the user won't be able to `std::cout << std::uint_least128_t{...}`, however, the language provides sufficient alternatives (`std::printf`, `std::format`, `std::printf, out << std::to_string(...)`).

Issue: `std::printf` and `std::scanf` need to support 128-bit integers.

Action: ✓ None (see § 8.2.17 `<cstdio>` for implementation experience).

Issue: `<cinttypes>` needs to include the wording changes for `<inttypes.h>`.

Action: ⚠ Include changes (see § 9.8 Header `<cinttypes>` for wording).

§ 4.3.12. Concurrency support library

Issue: `std::atomic` needs to support `std::uint_least128_t`.

Action: ✓ No impact on the standard (see § 8.2.18 `std::atomic` for implementation experience).

Issue: There should be additional aliases `std::atomic_uint_least128_t` et al. aliases.

Action: ⚠ Define aliases (see § 9.9 Atomic operations for wording).

§ 5. Impact on implementations

§ 5.1. Estimated implementation effort

The following table summarizes the affected standard library parts and the estimated effort required to implement the proposed changes.

Affected library part	Work definitely required	Implementation experience
<code>std::to_integer</code>	✓ no	§ 8.2.1 <code>std::to_integer</code>
<code><version></code>	§ 9.1 Header <code><version></code>	
<code><limits></code>	add specializations	
<code><climits></code>	add macro constants	
<code>std::is_integral</code>	✓ no	§ 8.2.2 <code><type_traits></code>
<code>std::make_{un}signed</code>	✓ no	§ 8.2.2 <code><type_traits></code>
<code>std::cmp_xxx</code>	✓ no	§ 8.2.3 <code>std::cmp_xxx</code>
<code>std::integer_sequence</code>	✓ no	
<code><bitset></code>	§ 9.4 Class template <code>bitset</code>	§ 8.2.4 <code><bitset></code>
<code><charconv></code>	✓ no	§ 8.2.5 <code><charconv></code>
<code><format></code>	✓ no	§ 8.2.6 <code><format></code>
<code><bit></code>	support 128-bit	§ 8.2.7 <code><bit></code>
<code>std::to_string</code>	§ 9.5 Numeric conversions	§ 8.2.8 <code>std::to_string</code>
<code>std::iota_view</code>	§ 9.6 Iota view	§ 5.2 <code>std::ranges::iota_view</code> ABI issue
<code>std::gcd</code> , <code>std::lcm</code>	✓ no	§ 8.2.9 <code>std::gcd</code> , <code>std::lcm</code>
<code>std::midpoint</code>	✓ no	§ 8.2.11 <code>std::midpoint</code>
<code>std::xxx_sat</code>	✓ no	§ 8.2.12 <code>std::xxx_sat</code>
<code><random></code>	256-bit LCG	§ 8.2.10 <code><random></code>
<code>std::valarray</code>	✓ no	§ 8.2.15 <code>std::valarray</code>
<code><cmath></code> overloads	✓ no	§ 8.2.14 <code><cmath></code>
<code>std::abs</code>	§ 9.7 Absolute values	§ 8.2.13 <code>std::abs</code>
<code><linalg></code>	✓ no	§ 8.2.16 <code><linalg></code>
<code>std::printf</code> , <code>std::scanf</code>	✓ no	§ 8.2.17 <code><cstdio></code>
<code><inttypes.h></code>	§ 9.3 Header <code><inttypes.h></code>	
<code><stdintypes></code>	§ 9.8 Header <code><stdintypes></code>	

When deciding "*Work definitely required*", this paper does not consider menial changes like relaxing `static_assert(__is_standard_integer<T>)` and such, which may be present in functions such as `std::gcd`.

Also, if there exists at least one standard library which implements these features, it is assumed that they can be adapted into other libraries with relative ease.

§ 5.2. std::ranges::iota_view ABI issue

libstdc++ defines `difference_type` for `std::ranges::iota_view<long long>` to be `__int128`. Since a `std::int_least128_t` alias would likely be defined as `__int128`, there is no ABI impact. Other libraries are not so fortunate.

§ 5.2.1. Affected implementations

By contrast, the `difference_type` for `std::ranges::iota_view<long long>` in libc++ is `long long`.

The MSVC STL uses a class type `std::_Signed128`. Even trivially copyable classes aren't passed via register in the Microsoft x86_64 ABI, so this type is passed via the stack. Re-defining this to be an integer type would break ABI, assuming that a Microsoft `__int128` would be passed via registers.

§ 5.2.2. Cause of ABI break

The ABI break stems from the fact that $IOTA-DIFF-T(W)$ for $W = \text{long long}$ is defined to be:

a signed integer type of width greater than the width of W if such a type exists.

Currently, no such type exists, but if `std::int_least128_t` did exist, it would no longer be valid to use a class type or `long long` as a `difference_type`.

§ 5.2.3. Possible solution

See § 9.6 Iota view for a proposed solution which resolves this issue without requiring action from implementors.

§ 5.2.4. What about `iota_view<std::int_least128_t>`?

Besides the option to provide a ≥ 129 -bit `difference_type`, implementations can also define `difference_type` to be a 128-bit integer.

Neither the *Cpp17RandomAccessIterator* requirement nor the `random_access_iterator` concept require the difference between two iterators to be representable using their `difference_type`. Therefore, this is a reasonable strategy which is easy to implement. Of course, it has the adverse affect that `a - b` is possibly undefined behavior for two iterators.

In practice, will the user ever need a 128-bit `iota_view`, and if so, do they need to represent such extreme differences? These are quality-of-implementation issues which maintainers will need to consider. libstdc++ already supports `std::iota_view<__int128>`, where the `difference_type` is `__int128`. Besides QoI questions, this proposal does not introduce any new issues.

§ 6. Impact on existing code

With no core language changes and only additional standard library features, the impact on existing code should be minimal.

§ 6.1. Possible semantic changes

However, this idea is put into question when it comes to integer literals.

EXAMPLE 11

```
auto x = 18446744073709551615; // 264-1
```

If the widest signed integer type is a 64-bit type, this code is ill-formed. Every compiler handles this differently:

- clang declares `x` as `unsigned long long` and emits a warning (✓ compliant).
- GCC declares `x` as `__int128` and emits a misleading warning (✓ compliant).
- MSVC declares `x` as `unsigned long long` and emits no warning (✗ non-compliant).

The example demonstrates that in practice, introducing a 128-bit integer may impact some existing code. To comply with the C++ standard, the type of `18446744073709551615` would have to be

`std::int_least128_t` assuming that `long long` cannot represent the value.

Hexadecimal literals are not affected in the same way because they are required to be of type `unsigned long long` if `long long` cannot represent the value. The introduction of a 128-bit integer type would not alter the signedness of existing literals.

§ 6.2. Impact on overload sets

Besides templates, a popular technique for covering multiple integer types is to create an "exhaustive" overload sets like:

```
// support "all" signed integers (anything less than int is promoted)
void foo(int);
void foo(long);
void foo(long long);
```

I'm putting "exhaustive" in quotes because such code does not cover extended integer types, which can exist. Only the implementation knows the whole set of integer types and can ensure completeness.

NOTE: Creating an overload set from `std::int8_t`, `std::int16_t`, `std::int32_t`, and `std::int64_t` is not possible because it only covers four out of five standard integer types, making some calls ambiguous.

While creating sets like these outside of language implementations is not ideal, the proposal can minimize the impact by making `std::int_least128_t` a distinct type from standard integer types.

EXAMPLE 12

If `std::int_least128_t` is `long long`, the following code is ill-formed:

```
void foo(int) { }
void foo(long) { }
void foo(long long) { }
void foo(std::int_least128_t) { } // re-definition of foo(long long)
```

There exists no implementation where `long long` is 128-bit, so no code is really affected.

NOTE: A workaround is to write `void foo(std::same_as<std::int_least128_t> auto)`. However, this solution is not obvious, and should not be necessary.

§ 6.2.1. Proposed solution

There should exist a natural and universally correct way to extend such overload sets, so that the effort of "upgrading" to 128-bit is minimal. Therefore `std::int_least128_t` should be distinct.

Guaranteeing that `std::int_least128_t` is distinct means that even if `long long` is a 128-bit type, it won't be chosen by this alias. This breaks the conventions of `<cstdint>` and may be surprising, but no implementation with `<cstdint>` aliases beyond 64 exists, and no implementation where `long long` is 128-bit exists. No existing code is affected; this is a purely academic problem.

§ 6.3. Possible assumption violations

There is obviously a substantial amount of code which assumes that integers are no wider than 64 bits. There is also a substantial amount of code which assumes that `std::intmax_t` is the widest integer type, and this assumption would be broken by introducing `std::uint_least128_t`.

The exact impact is investigated in this proposal. Assumptions about hardware or integer width limitations cannot hold back language development. C would be stuck with 32-bit types if that had ever been a convincing rationale. Also, the introduction of a 128-bit type does not break existing code unless the user chooses to use it.

§ 6.4. `std::bitset` constructor semantic changes

The only overload which accepts integers is `bitset(unsigned long long)`. Ideally, we would like to construct bitsets from wider integer types, if available. **My proposed solution changes the semantics of this constructor** (see § 9.4 Class template `bitset` for wording).

The existing constructor is problematic for multiple reasons:

1. If extended by a `std::int_least128_t` overload, a call `bitset<N>(0)` would become ambiguous.
2. When called with negative numbers, a sign extension only takes place up to the width of `unsigned long long`. Beyond that, the bits are zero-filled.

EXAMPLE 13

The following assertion passes:

```
#include <bitset>
constexpr std::bitset<128> bits(-1);
static_assert(bits.count() == 64);
```

The original behavior is very difficult to preserve if we add more overloads. If we added an `int_least128_t` overload, then `bitset(0)` would be ambiguous. Therefore, we must at least have an overload for all integers with a conversion rank of `int` or greater.

However, if so, `-1` under the current definition would result in a `std::bitset<128>` that has only 32 one-bits (assuming 32-bit `int`). We could preserve the current behavior exactly if sign-extension occurred up to the width of `unsigned long long`; beyond that, zero-extension would be used. This is not proposed because the design makes no sense outside of its historical context.

§ 6.4.1. Proposed solution

Therefore, I propose to perform sign-extension for the full size of the `bitset`. In other words, `std::bitset<N>(-1)` would always be a bitset where every bit is set. This almost certainly matches the intent of the user.

A GitHub code search for `/bitset<.*>\(-[0-9]+\)/ language:c++` finds 30 uses of constructing a bitset from a negative literal. Of the ones which use `std::bitset`, all uses are of the form

- `std::bitset<N>(-1)` where `N` is less than 64, or
- `std::bitset<N>(static_cast<T>(-1))`.

None of these existing uses would be affected.

NOTE: See § 9.4 Class template `bitset` for wording.

§ 7. Design considerations

The goal of this proposal is to obtain a mandatory 128-bit type with strong library support. A `std::least_uint128_t` alias is the only option that does not involve any changes to the core

language. Therefore, it is the obvious design choice for this proposal.

NOTE: Unlike the existing `int_leastN_t` and `int_fastN_t` aliases, this type is distinct. See [§ 6.2 Impact on overload sets](#) for rationale, and [§ 9.2 Header `<cstdint>`](#) for wording.

Besides the current approach, there are a few alternatives which have been considered:

§ 7.1. Why no standard integer type?

Why standardize a `std::uint_least128_t` type alias but no standard integer type? Essentially, why no unsigned long long long?

Firstly, naming is a problem here. A standard integer type would likely warrant the ability to name it by keyword, and an ever-increasing sequence of `longs` isn't an attractive solution. Even with a concise keyword such as `_Uint128`, it is unclear what advantage such a keyword would have over a type alias, other than saving one `#include <cstdint>` directive.

Secondly, it is useful to keep `std::uint_least128_t` a second-class citizen by not making it a standard integer type. For example, in the formatting library, a format string can specify a dynamic *width* for an argument, which must be a standard integer. A *width* that cannot be represented by a 64-bit number is unreasonable, so it makes sense to limit support to standard integers.

Thirdly, as already stated in [§ 4.1 C Compatibility](#), C23's `intmax_t` must be the widest standard integer type. To not break ABI and be C23-compatible, `std::int_least128_t` must be an extended integer type.

§ 7.2. Why no mandatory `std::int128_t` type?

Mandating any exact `std::intN_t` inadvertently restricts the byte width because exact-width types cannot have any padding. `std::int128_t` implies that the width of a byte is a power of two ≤ 128 , and historically, C++ has not restricted implementations to a specific byte size.

This decrease in portability also has no good rationale. If `std::int_least128_t` is mandatory and an implementation is able to define it without padding, then `std::int128_t` is effectively mandatory.

Hypothetically, a malicious implementation could define `std::int_least128_t` to be a 1000-bit integer with 872 padding bits, even if it was able to define a padding-free 128-bit integer. However,

malicious implementations have never been a strong argument to guide design.

§ 7.3. Why no `std::int_least256_t` type?

256-bit integers are also useful and one could use many of the arguments in favor of 128-bit integers to also propose them. However, there are a few strong reasons against including them in this proposal:

1. The wider the bit sizes, the fewer the use cases are. For example, 128 bits are sufficient for high-precision clocks and most financial applications.
2. There is tremendously less hardware support for 256-bit integers. x86 has instructions to perform a 64-to-128-bit multiplication, but no such 128-to-256-bit instruction exists.
3. There are fewer existing implementations of 256-bit integers.
4. Many use cases of 256-bit integers are simply bit manipulation. The wider the type, the less common arithmetic becomes. Bitwise operations (`&`, `|`, `~`) are best done through vector registers, but the ABI for `_BitInt(256)` is to use general purpose registers or the stack, and `std::int_least256_t` would likely be the same. Since there is strong hardware support for § 3.1.3 [Widening operations](#) which is based on general purpose registers, the choice for 128-bit is easy; not so much for 256-bit.
5. `std::uint128_t` is a "support type" for `std::float128_t` which simplifies the implementation of many `<cmath>` functions. There exists no hardware with *binary256* support to motivate a `std::uint256_t` support type.
6. Longer integer literals are a major reason to get a fundamental type. 256-bit hexadecimal literals are up to 64 digits long, which degrades code quality too much. With indentation and digit separators, using the full width can exceed the auto-formatter's column limit.

It is also unclear whether there should ever be a mandatory 256-bit extended integer type, or if support should be provided through 256-bit bit-precise integers. Overall, this proposal is more focused if it includes only 128-bit.

Nevertheless, many of the changes in § 9 [Proposed wording](#) pave the way for a future `std::int_least256_t` or even `std::int_least512_t`. There would be no wording impact other than defining the necessary aliases and macros.

§ 7.4. Why no class type?

Instead of an extended integer type, it would also be possible to provide the user with a 128-bit class type. This could even be done through a general `std::big_int<N>` class. However, there are compelling reasons against doing so:

1. `std::int_least128_t` is sufficiently common to where the added cost of class type (overload resolution for operators, function call evaluation in constant expressions, etc.) would be a burden to the user. `__int128` is the "work horse" of any multi-precision library which uses 64-to-128-bit widening operations (see § 3.1.3.2 [Multi-precision operations](#)). This cost could add up quickly.
2. A fundamental type also comes with integer literals, and up to 128-bit, there are still reasonable use cases for integer literals. § 3 [Motivation and scope](#) shows multiple examples where 128-bit literals were used. Besides these use cases, it would be nice to represent an IPv6 address using a hexadecimal literal (which is the typical representation of these addresses).
3. There are 128-bit architectures where the general purpose register size is 128-bit. For example, the RV128I variant of RISC-V is such an architecture. To be fair, there exists no implementation of RV128I yet. Still, it would be unusual not to have a fundamental type that represents the general purpose register.

In essence, 128-bit is still "special" enough to deserve a fundamental type. Beyond 128-bit, even hexadecimal literals become hard to read due to their sheer length, and we are unlikely to find any ISA with a 256-bit register size, even counting theoretical ISAs like RV128I.

§ 7.5. Why no bit-precise integers?

Instead of putting work into 128-bit integers, it would also be possible to integrate bit-precise integers (C's `_BitInt(N)` type, proposed in [\[N2763\]](#)) into the C++ standard.

This would be a sufficient alternative if `_BitInt` was a fundamental type, had integer literal support, and had strong library support. However, there are numerous reasons why `_BitInt` is not the right path for this proposal, described below. In short, this proposal argues that `_BitInt` does not bring sufficient value to C++ relative to its impact, and would better be exposed via a class type `std::big_int` than a fundamental type.

§ 7.5.1. `_BitInt` has no existing C++ work

After enquiry in the std-proposals mailing list, no one has expressed that they are working on `_BitInt`, nor has anyone expressed interest on beginning work on this. Right now, `_BitInt` is a

purely hypothetical feature.

§ 7.5.2. `_BitInt` has less motivation in C++

A significant part of the rationale for [N2709] was that only `_BitInt` can utilize hardware resources optimally on hardware such as FPGAs that have a native 2031-bit type. C++ is much less ambitious in its goal of supporting *all* hardware, with changes such as C++20 effectively mandating two's complement signed integers. C23 supporting `_BitInt` is not rationale for a C++ fundamental type in itself.

Therefore, a limited solution as focusing on 128-bit is not unreasonable.

§ 7.5.3. `_BitInt` should be exposed as a class type

By default, all new language features are library features. If it's possible to express N-bit integers through a class, then this is the go-to solution.

In C++, it is possible to expose the compiler's `_BitInt` functionality as follows:

```
inline constexpr size_t big_int_max_width = BITINT_MAXWIDTH;

template <size_t N>
    requires (N <= BITINT_MAXWIDTH)
struct big_int {
    _BitInt(N) _M_value;

    // TODO: constructors, operator overloads, etc.
};

// TODO: define big_uint similarly

// compatibility macros:
#ifdef __cplusplus
#define _BigInt(...) ::std::big_int<__VA_ARGS__>
#define _BigUInt(...) ::std::big_uint<__VA_ARGS__>
#else
#define _BigInt(...) _BitInt(__VA_ARGS__)
#define _BIgUInt(...) unsigned _BitInt(__VA_ARGS__)
#endif
```

There is precedent for this design:

- `std::complex<T>` is compatible with C's `_Complex T`.
- `std::atomic<T>` is compatible with C's `_Atomic(T)`.

Unless an extremely strong case for a new category of integer types in the core language can be made, this is the obvious solution.

Of course, a class type is more costly in terms of compilation slowdown as well as performance on debug builds and in constant evaluations. This is acceptable because `_BitInt` is not a replacement for the existing integers, but an infrequently used special-purpose type which only comes into play when no other size would suffice.

§ 7.5.4. `_BitInt` is not a replacement for standard integers

In C, `_BitInt` is a second-class citizen. The overwhelming majority of library functions take `int`, `long`, and other standard integer types. Conversion rules are also biased in favor of standard integers. For example, `_BitInt(32) {0} + (int) {0}` is of type `int` in C, if `int` is 32-bit.

Standard integers are essentially engraved into the language and have special status, both in C and in C++. Billions of lines of code use these integers, and this is never going to change. Virtually all learning resources in existence teach standard integers or `<stdint>` aliases, which are standard integers in all implementations.

Even if `_BitInt` was a replacement for the existing integers, the transition process would take a century. It is better to think of it as an extension of the existing integers.

§ 7.5.5. `_BitInt` does not guarantee 128-bit support

The whole point of this proposal is to guarantee developers a 128-bit type. However, `_BitInt(N)` is only valid for `N <= BITINT_MAXWIDTH`, where `BITINT_MAXWIDTH` is guaranteed to be no more than the width of `long long`.

§ 7.5.6. `_BitInt` requires more library effort

This proposal only mandates `std::int_least128_t`, not any specific width. Therefore, implementers have the luxury of relying on all integers being padding-free and all integers having a width which is 2^N times the platform's byte size.

These assumptions greatly simplify the implementation of various algorithms. From personal experience, implementing any functions in `<bit>` is tremendously easier when it is guaranteed that integers have a 2^N width.

Full `_BitInt` library support requires tremendously greater library effort. It is unclear what parts of the standard library can be burdened.

§ 7.5.7. `_BitInt` breaks more existing code

`_BitInt` also breaks assumptions in existing, generic code. C++ users have enjoyed the luxury of padding-free integers on conventional implementations for a very long time, and some code depends on it.

It may depend through anti-patterns like using `memcmp` for comparison of `struct`s storing integers, or more justified uses like `std::bit_cast<T>(integer)`. `_BitInt` inevitably breaks assumptions about padding and size, which is a great challenge to both the implementation and C++ users.

EXAMPLE 14

The following pseudo-code has undefined behavior if `_BitInt(30)` has padding bits.

```
template <std::integral T>
auto to_byte_array(const T& x) {
    return std::bit_cast<std::array<std::byte, sizeof(T)>>(x);
}

int main() {
    _BitInt(30) x = ...;
    write_buffer_to_file(file, to_byte_array(x));
}
```

Assuming a 4-byte array is returned, `std::bit_cast` one of these bytes would have indeterminate value because the corresponding byte in `_BitInt(30)` has padding bits. Reading this indeterminate value to store it in a file is undefined behavior.

While the assumption that all integers are padding-free is not universally correct, C++ users have enjoyed this guarantee for decades, and an unknown amount of code depends on it. If `_BitInt` was just another integral type, it could silently break existing code like in the example, despite a `std::integral` constraint.

NOTE: `std::int_least128_t` does not have the same problem because the implementation can define it as a type with more than 128 bits which has no padding, if need be.

§ 7.5.8. `_BitInt` has teachability issues

`_BitInt` has different promotion and conversion rules. These rules are not necessarily obvious, especially when bit-precise integers interact with other types.

For example, `x + 1` is of type `int` if `x` is of type `_BitInt(32)` or narrower, and `int` is 32-bit. Not only does the user have to learn the existing integer conversion and promotion rules, the user also has to learn this new `_BitInt` system and how it interacts with the old system. This also complicates overload resolution:

EXAMPLE 15

```
void foo(int);
void foo(_BitInt(32));

int main() {
    _BitInt(32) x = 0;
    foo(x); // calls which?
}
```

If `int` dominates in implicit conversions, should it also dominate in overload resolution so that `foo(int)` is selected? The answer is not entirely clear.

EXAMPLE 16

```
void foo(int);
void foo(_BitInt(32));

int main() {
    foo(1'000'000'000); // calls which?
}
```

On a target where `int` is 16-bit, the integer literal is of type `long`. It is possible to convert it to `int` in a narrowing conversion, and possible to convert it to `_BitInt(32)` losslessly.

- Should overload resolution prefer the lossless conversion to `_BitInt(32)`? After all, lossless is better.
- Should the call be ambiguous? After all, that's what normally happens when there are two viable candidates with equally long conversion sequences.
- Should `foo(int)` be called? After all, standard integers are normally dominant, and `1'000'000'000` could be of type `int` if only `int` was 32-bit.

There is a compelling case for each of these options, and no matter what design choice is made, the complexity of the language increases significantly.

§ 7.5.9. `_BitInt` in C may be too permissive

Many C++ users have lamented that integers are too permissive. As is tradition, C has not restricted the behavior of `_BitInt` substantially:

- Signed/unsigned mixed comparison are permitted.
- Signed/unsigned mixed arithmetic is permitted.
- Implicit truncating conversion is permitted.
- Implicit signed/unsigned conversion is permitted.

One substantial difference is that `_BitInt` is not promoted to `int`, other than that, the semantics are the same as the old integers. If `_BitInt` behaves almost the same as standard integers and brings all of this bug-prone behavior with it, how can one justify adding it as a new fundamental type?

Of course, C++ could decide to make these semantics more restrictive for its `_BitInt` type, similar to how implicit casts from `void*` are only permitted in C, but not in C++. However, this would complicate writing C/C++ interoperable code and make the language even less teachable because `_BitInt` semantics would become language-specific.

Furthermore, the more distinct the `_BitInt` semantics become, the less of a drop-in replacement for `__int128` it becomes:

EXAMPLE 17

The following function yields the high 64 bits of a multiplication. Similar code can be found in [Stockfish] (see also § 3.1.3 Widening operations).

```
std::uint64_t mul_hi(std::uint64_t x, std::uint64_t y) {  
    return u128(x) * u128(y) >> 64;  
}
```

If `u128` is extended integer type, this code is well-formed. If `u128` is a C-style `unsigned _BitInt(128)`, this code is well-formed. If `u128` is a more restrictive C++ `unsigned _BitInt(128)` which forbids narrowing, this code is ill-formed.

In short, the dilemma is as follows:

- If `_BitInt` is as permissive as existing integers, it is harder to justify its addition to the language because it innovates too little.
- If `_BitInt` is more restrictive, its impact on existing code and on language teachability is greater.

There is no obvious path here, only a bottomless potential for discussion. By comparison, `std::int_least128_t` has exactly the same rules as existing integers, which `__int128` follows. The user can use it as a drop-in replacement:

```
#ifdef __SIZEOF_INT128__  
using i128 = __int128 std::int128_t;  
#else  
// struct i128 { /* ... */ };  
#endif
```

§ 7.5.10. `_BitInt` false dichotomy

The `_BitInt` vs. `std::int_least128_t` argument is a false dichotomy. Bit-precise integers essentially create a parallel, alternative type system with different rules for promotion, implicit conversion, and possibly overload resolution.

Defining any `<cstdint>/stdint.h` type alias as a bit-precise integer would be hugely surprising to language users, who have certain expectations towards aliases in these headers. These expectations have been formed over the past 30 years.

Therefore, even if all issues regarding `_BitInt` mentioned in this paper were resolved and `_BitInt` becomes a fundamental type, it would be reasonable to maintain the "legacy" type system in parallel.

§ 7.6. Why not make it optional?

Instead of making `std::int_least128_t` entirely mandatory, it would also be possible to make it an optional type, or to make it mandatory only on hosted implementations.

First and foremost, making the type optional has severe consequences. Library authors still have to write twice the code: one version with 128-bit support, one version without. To C++ users, the ability to write portable code is more valuable than the underlying implementation effort, or potential performance issues. I will go into these two issues more:

§ 7.6.1. Implementation effort is not too high

The criticism is:

On freestanding/embedded platforms, the implementation effort of `std::int_least128_t` is too great.

While this concern is valid, C23 requires arbitrary-precision arithmetic through `_BitInt` anyway and GCC and clang support `_BitInt(128)` already (see § 8.1.4 `_BitInt(128)` for support). Assuming that vendors care about C-compatibility, this proposal merely requires vendors to provide `int_least128_t = _BitInt(128)`.

NOTE: Bit-precise integers have slightly different semantics than extended integers. However, these differences don't matter if `_BitInt(128)` is the widest integer, making it valid to use as `int_least128_t`.

The remaining impact is limited to the standard library. For the most part, it is simple to generalize library algorithms to an arbitrary bit size. It is especially easy when the implementation can ensure that all integers are padding-free and have a size that is a 2^N multiple of the byte size. Only `int_leastN_t` is mandatory (not the exact-width types), so the implementation can ensure it.

§ 7.6.2. Software emulation is acceptable

The criticism is:

`std::int_least128_t` should not be mandatory if software emulation degrades performance.

There is also merit to this concern. A mandatory type may give the user a false sense of hardware support which simply doesn't exist, especially on 32-bit or even 8-bit hardware.

However, this problem is innate to standard integers as well. If a user is compiling for a 32-bit architecture, a 64-bit `long long` will have to be software-emulated, and 64-bit integer division can have dramatic cost. Why should a 64-bit `long long` be mandatory on an 8-bit architecture? The answer is: because it's useful to rely on `long long` so we can write portable code, even if we try to avoid the type for the sake of performance.

In the end, it's the responsibility of the user to be vaguely aware of hardware capabilities and not use integer types that are poorly supported. If the user wants to perform a 128-bit integer division on an 8-bit machine, the language shouldn't artificially restrict them from doing so. The same principle applies to `long long`, `std::int_least128_t`, C23's `_BitInt(1024)`, etc.

§ 7.7. Should extended integer semantics be changed?

An interesting question is whether extended integer semantics make sense in the first place, or require some form of changes. The relevant standard section is 6.8.6 [conv.rank].

In summary:

Relevant Rule	Example
Unsigned integers have the same rank as signed integers of equal width.	<code>std::uint128_t == std::int128_t</code>
Wider integers have greater rank.	<code>std::uint128_t == std::int64_t</code>
Standard integers of equal width have greater rank.	<code>long long > std::int128_t</code> if <code>long long</code> is 128-bit

The shortest user-defined literal that does not fit into `unsigned long long` is `0xffffffffffffffffffffs`, which is 20 characters long. At this length, user-defined literals never provide overwhelming utility.

However, if WG21 favors new forms of `operator""` for integer types wider than `unsigned long long`, I am open to working on this.

NOTE: The standard currently does not allow `operator""_zero(integer)` for any `integer` except `unsigned long long`.

§ 7.9. Why no `std::div`?

`std::div` is a function which returns the quotient and remainder of an integer division in one operation. This proposal intentionally doesn't extend support to 128-bit types because each overload of `std::div` returns a different type. Namely, the current overloads for `int`, `long`, and `long long` return `std::div_t`, `std::ldiv_t`, and `std::lldiv_t` respectively.

This scheme isn't easy to generalize to 128-bit integers or other extended integer types. A possibly approach would be to define a class template `std::div_result<T>` and re-define the concrete types to be aliases for `std::div_result<int>` etc. However, this is arguably a breaking change because it alters what template argument deduction is possible from these types.

Furthermore, `std::div` is arguably useless. Optimizing compilers recognize separate uses of `x / y` and `x % y` and fuse them into a single division which yields both quotient and remainder, at least on platforms where this is possible.

NOTE: In C89, `div` was useful because it had a well specified rounding mode, whereas the division operator had implementation-defined rounding.

§ 7.10. What about the `std::ratio` dilemma?

Assuming that C++26 is based on C23, `std::ratio` will be problematic because it is defined as:

```
template<intmax_t Num, intmax_t Denom = 1>
class ratio { /* ... */ };
```

`std::intmax_t` would no longer be the widest integer type, and certain extreme ratios would become unrepresentable. It is not possible to redefine it to have other types for template parameters because the types of template arguments participate in name mangling.

This issue is not caused by this proposal, but the introduction of a 128-bit integer first manifests it.

This proposal does not attempt to resolve it. However, a possible path forward is to make `std::duration` less dependent on `std::ratio` and allow *ratio-like* types instead.

§ 8. Implementation experience

§ 8.1. Existing 128-bit integer types

§ 8.1.1. `__int128` (GNU-like)

GCC and clang already provide the 128-bit integer types in the form of `__int128` and `unsigned __int128`. However, this type is not available when compiling for 32-bit targets. Clang provides the same support.

§ 8.1.2. `__int128` (CUDA)

In NVIDIA CUDA 11.5, the NVCC compiler has added preview support for the signed and unsigned `__int128` data types on platforms where the host compiler supports it. See [\[NVIDIA\]](#).

§ 8.1.3. `std::_Signed128`, `std::_Unsigned128`

The MSVC STL provides the class types `std::_Signed128` and `std::_Unsigned128` defined in `<__msvc_int128.hpp>`. These types implement all arithmetic operations and integer comparisons.

They satisfy the *integer-like* constraint and have been added to implement [\[P1522R1\]](#). `std::iota_view::difference_type` is possibly defined as `std::_Signed128`.

§ 8.1.4. `_BitInt(128)`

The C23 standard requires support for bit-precise integers `_BitInt(N <= BITINT_MAXWIDTH)` where `BITINT_MAXWIDTH >= ULLONG_WIDTH`. While this doesn't strictly force support for 128-bit integers, GNU-family implementations support more than 128 bits already.

As of February 2024, the support is as follows:

Compiler	<code>__BITINT_MAXWIDTH</code>	Targets	Languages
clang 14	128	all	C & C++
clang 16	8388608	all	C & C++
GCC 14	65535	64-bit only	C
MSVC 19.38	✗	✗	✗

NOTE: clang has supported `__BitInt` as an `__ExtInt` compiler extension prior to C standardization.

It is possible that given enough time, `__BitInt(128)` will be supported by Microsoft as well.

NOTE: Microsoft Developer Community users have requested support for a 128-bit type at [\[MSDN\]](#).

§ 8.2. Library implementation experience

§ 8.2.1. `std::to_integer`

`std::to_integer<T>(...)` is equivalent to `static_cast<T>(...)` with constraints.

§ 8.2.2. `<type_traits>`

Assuming that `is_integral` and `make_{un}signed` don't simply delegate to a compiler intrinsic, implementing these traits merely requires adding two specializations such as `is_integral<int_least128_t> : false_type`.

See libstdc++'s `<type_traits>`.

§ 8.2.3. `std::cmp_XXX`

Libstdc++ provides a width-agnostic implementation of `std::cmp_equal` and other safe comparison functions in `<utility>`.

Being able to extend to a wider type is helpful in principle (e.g. implementing `std::cmp_equal(int, int)` in terms of a comparison between `long`s), however, the current

implementations don't make use of this opportunity anyway.

§ 8.2.4. `<bitset>`

NOTE: See § 9.4 Class template `bitset` for proposed changes.

To implement these changes, a constructor and member function template can be defined:

```
bitset(integral auto);  
  
template <unsigned_integral T>  
T to();
```

These are functionally equivalent to the existing `unsigned long long` constructor and `to_ullong` function respectively, just generalized.

§ 8.2.5. `<charconv>`

libstdc++ already provides a width-agnostic implementation of `std::to_chars` in `<bits/charconv.h>`, and a width-agnostic implementation of `std::from_chars` in `<charconv>`.

In general, it is not difficult to generalize `std::to_chars` for any width. Stringification uses integer division, which may be a problem. However, the divisor is constant. Due to strength reduction optimization (see § 3.1.4 Fixed-point operations for an example), no extreme cost is incurred no matter the width.

§ 8.2.6. `<format>`

libstdc++ already supports `std::format` for `__int128`.

The locale-independent forms are simply implemented in terms of `std::to_chars` and are not affected by the introduction of 128-bit integers. As explained above, `std::to_chars` implementations typically already support 128-bit integers.

The new `std::basic_format_parse_context::check_dynamic_spec` function is not affected. This function only checks for the type of a dynamic *width* or *precision*, and the arguments are required

to be of standard integer type. Realistically the user will never need a 128-bit *width* or *precision*, which is why no changes are proposed.

`std::basic_format_arg` also requires no changes because `std::basic_format_arg::handle` already covers extended integer and floating-point types. Also, modifying the `value variant` within a `std::basic_format_arg` would be an avoidable ABI-break.

§ 8.2.7. `<bit>`

In [\[BitPermutations\]](#), I have implemented the majority of C++ bit manipulation functions for *any* width, i.e. in a way that is compatible with `_BitInt(N)` for any `N`.

Such an extremely generalized implementation is challenging, however, merely extending support to 128-bit given a 64-bit implementation is simple.

EXAMPLE 19

Given a 64-bit `std::popcount`, a 128-bit implementation looks as follows:

```
int popcount(uint128_t x) {  
    return popcount(uint64_t(x >> 64)) + popcount(uint64_t(x));  
}
```

EXAMPLE 20

Given a 64-bit `std::countr_zero`, a 128-bit implementation looks as follows:

```
int countr_zero(uint128_t x) {  
    int result = countr_zero(uint64_t(x));  
    return result < 64 ? result : 64 + countr_zero(uint64_t(x >> 64));  
}
```

All bit manipulation functions are easily constructed this way.

§ 8.2.8. `std::to_string`

NOTE: See [§ 9.5 Numeric conversions](#) for proposed changes.

libstdc++ already implements `to_string` as an inline function which forwards to `detail::__to_chars_10_impl`.

In general, `std::to_string` simply needs to forward to `std::to_chars` or a similar function, and this is easily generalized.

§ 8.2.9. `std::gcd`, `std::lcm`

libstdc++ provides a `std::gcd` implementation which uses the [Binary GCD algorithm](#). The MSVC STL has a similar implementation. This algorithm is easily generalized to any width. It requires `std::countr_zero` for an efficient implementation, which is easy to implement for 128-bit integers (see § 8.2.7 <bit>).

libc++ uses a naive `std::gcd` implementation based on the Euclidean Algorithm, which relies on integer division. Due to the immense cost of integer division for 128-bit integers, such an implementation may need revision.

`std::lcm` requires no work because mathematically, $\text{gcd}(x, y) * \text{lcm}(x, y) == x * y$. When solving for `lcm`, $\text{lcm}(x, y) = x / \text{gcd}(x, y) * y$. The implementation effort (if any) is limited to `std::gcd`.

NOTE: By dividing by `gcd(x, y)` prior to multiplication, overflow in `x * y` is avoided. Overflow can only occur if `lcm(x, y)` is not representable by the result type.

§ 8.2.10. <random>

`std::linear_congruential_engine<T, a, c, m>` requires at least double-wide integers to safely perform the operation $(a * x + c) \bmod m$, where `x` is the LCG state. Otherwise, the multiplication and addition could overflow.

libstdc++ solves this issue by performing all operations using `__int128` if available (see [<bits/random.h>](#)), and otherwise:

```
static_assert(__which < 0, /* needs to be dependent */
              "sorry, would be too much trouble for a slow result");
```

Introducing 128-bit integers would force implementations to also provide 256-bit operations solely for the purpose of `std::linear_congruential_engine`. This can be considered reasonable because

C23 requires implementations to provide arbitrary-precision arithmetic anyway, and both GCC and clang already implement `_BitInt(N)` for `N >= 256` (see § 8.1.4 `_BitInt(128)` for details on support).

§ 8.2.11. `std::midpoint`

Libstdc++ has a `std::midpoint` implementation which is width-agnostic.

§ 8.2.12. `std::xxx_sat`

libstdc++ provides a width-agnostic implementation for all saturating arithmetic functions in `<bits/sat_arith.h>`.

Saturating arithmetic is generally done through compiler intrinsics such as `__builtin_mul_overflow`. These are already supported by GCC and Clang. A software implementation of overflow detection may be very tedious as explained in [P0543R3], but that isn't the chosen implementation anyway.

§ 8.2.13. `std::abs`

NOTE: See § 9.7 [Absolute values](#) for proposed changes.

`std::abs` can be easily implemented width-agnostically as `x >= 0 ? x : -x` for any integer `x`.

Note that an overload must exist for every integer type to avoid calling `std::abs` for floating-point types. Such an overload is proposed in § 9.7 [Absolute values](#).

§ 8.2.14. `<cmath>`

libstdc++, libc++, and the MSVC STL implement the integral math overloads using SFINAE. Effectively, they define function templates using `std::enable_if_t<std::is_integral_v<T>>`. Therefore, no changes are required.

§ 8.2.15. `std::valarray`

`std::valarray<T>` does not rely on any specific bit-size, or on `T` being any type in general. While it is possible to provide specializations for specific types that make more optimal use of hardware, it is also possible to rely on the optimizers auto-vectorization capabilities alone.

§ 8.2.16. `<linalg>`

The linear algebra library introduced by [P1673R13] does not rely on any specific widths and is generalized by default. The corresponding [reference implementation](#) can operate on `__int128`.

Providing specializations for specific widths is a quality-of-implementation issue.

§ 8.2.17. `<cstdio>`

NOTE: This proposal makes 128-bit `printf/scanf` support entirely optional (see § 9.3 Header `<inttypes.h>` for wording).

Similar to `std::to_chars`, extending support to 128-bit integers for printing and parsing requires only moderate effort because the underlying algorithm easily generalizes to any bit size. The `PRI*LEAST128` et al. macros in `<cinttypes>` would also need to be defined, and would expand to an implementation-defined format constant.

LLVM libc currently uses a `num_to_strview(uintmax_t, ...)` function for stringification. This would require replacement, possibly with a function template. Other standard libraries may be impacted more significantly.

NOTE: With the changes from [N2680] included, an alternative C23 way of printing 128-bit integers is:

```
std::printf("%w128d\n", std::int_least128_t{123});
```

§ 8.2.18. `std::atomic`

Libc++ already provides support for fetch-operations for `std::atomic<__int128>`. For example, `.fetch_add` delegates to `__atomic_fetch_add_16` in libatomic.

In general, the `fetch` operations that `std::atomic<long long>` provides must already have a software fallback for 32-bit hardware, where no 64-bit atomic `add` instruction exists. Such a software fallback may be implemented as a [CAS-and-retry](#) loop. The introduction of 128-bit integers adds no new challenges.

§ 9. Proposed wording

The proposed wording is relative to [\[CxxDraft\]](#), accessed 2024-02-10.

§ 9.1. Header `<version>`

In subclause 17.3.2 [version.syn], update the feature-testing macros as follows:

```
#define __cpp_lib_atomic_int128 20XXX
#define __cpp_lib_bitset 202306L20XXX
#define __cpp_lib_bitset_int128 20XXX
#define __cpp_lib_int128 20XXX
#define __cpp_lib_to_string 202306L20XXX
#define __cpp_lib_to_string_int128 20XXX
```

NOTE: Feature-detection for `std::printf` and `std::scanf` is intentionally omitted because the user can detect whether `PRI*LEAST128`, `SCN*FAST128` etc. are defined.

§ 9.2. Header `<cstdint>`

In subclause 17.4.1 [cstdint.syn], update the synopsis as follows:

```
// all freestanding
namespace std {
    using int8_t          = signed integer type;        // optional
    using int16_t         = signed integer type;        // optional
    using int32_t         = signed integer type;        // optional
    using int64_t         = signed integer type;        // optional
    using int128_t         = signed integer type;        // optional
    using intN_t          = see below;                  // optional

    using int_fast8_t     = signed integer type;
    using int_fast16_t    = signed integer type;
    using int_fast32_t    = signed integer type;
    using int_fast64_t    = signed integer type;
    using int_fast128_t   = signed integer type;
    using int_fastN_t     = see below;                  // optional

    using int_least8_t    = signed integer type;
    using int_least16_t   = signed integer type;
    using int_least32_t   = signed integer type;
    using int_least64_t   = signed integer type;
    using int_least128_t  = signed integer type;
    using int_leastN_t    = see below;                  // optional

    using intmax_t        = signed integer type;
    using intptr_t        = signed integer type;        // optional

    using uint8_t         = unsigned integer type;      // optional
    using uint16_t        = unsigned integer type;      // optional
    using uint32_t        = unsigned integer type;      // optional
    using uint64_t        = unsigned integer type;      // optional
    using uint128_t       = unsigned integer type;      // optional
    using uintN_t         = see below;                  // optional

    using uint_fast8_t    = unsigned integer type;
    using uint_fast16_t   = unsigned integer type;
    using uint_fast32_t   = unsigned integer type;
    using uint_fast64_t   = unsigned integer type;
    using uint_fast128_t  = unsigned integer type;
    using uint_fastN_t    = see below;                  // optional
```

```

using uint_least8_t    = unsigned integer type;
using uint_least16_t   = unsigned integer type;
using uint_least32_t   = unsigned integer type;
using uint_least64_t   = unsigned integer type;
using uint_least128_t  = unsigned integer type;
using uint_leastN_t    = see below;           // optional

using uintmax_t        = unsigned integer type;
using uintptr_t        = unsigned integer type; // optional
}

```

In subclause 17.4.1 [cstdint.syn], update paragraph 3 as follows:

All types that use the placeholder *N* are optional when *N* is not 8, 16, 32, ~~or 64~~ ,or 128 . The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, ~~and 64~~ ,and 128 are also optional; however, if an implementation defines integer types with the corresponding width and no padding bits, it defines the corresponding *typedef-names*. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name*.

In subclause 17.4.1 [cstdint.syn], add the following paragraph:

None of the types that use the placeholder *N* are standard integers types ([basic.fundamental]) if *N* is greater than 64.

[Example: `int_least128_t` is an extended integer type. `int_least64_t` is an extended integer type or a standard integer type whose width is at least 64. — end example]

NOTE: This restriction is intended to address § 6.2 [Impact on overload sets](#).

§ 9.3. Header `<inttypes.h>`

In subclause 17.14 [support.c.headers], add the following subclause:

17.14.X Header `<inttypes.h>`

The contents of the C++ header `<inttypes.h>` are the same as the C standard library header `<inttypes.h>` with the following exception: The definition of the `fprintf` and `fscanf` macros for the corresponding integers in the header `<stdint.h>` is optional for any integer with a width greater than 64. However, if any macro for an integer with width N is defined, all macros corresponding to integers with the same or lower width as N shall be defined.

See also: ISO/IEC 9899:2018, 7.8.1

NOTE: This effectively makes `printf/scanf` 128-bit support optional because without any `PRI/SCN` macros, the user has no standard way of using these functions with 128-bit integers.

NOTE: After rebasing on C23, additional restrictions to `stdio.h` must be applied so that `%w128d` (see [N2680]) is not mandatory in C++.

§ 9.4. Class template `bitset`

In subclause 22.9.2.1 [template.bitset.general], update the synopsis as follows:

```
// [bitset.cons], constructors
constexpr bitset() noexcept;
constexpr bitset(unsigned long long val) noexcept;
constexpr bitset(integer-least-int val) noexcept;
[...]
constexpr unsigned long      to_ulong() const;
constexpr unsigned long long to_ullong() const;
template<class T>
constexpr T to(.) const;
```

In subclause 22.9.2.1 [template.bitset.general], add a paragraph:

For each function with a parameter of type *integer-least-int*, the implementation provides an overload for each cv-unqualified integer type ([basic.fundamental]) whose conversion rank is that of `int` or greater, where *integer-least-int* in the function signature is replaced with that integer type.

NOTE: See § 6.4 [std::bitset constructor semantic changes](#) for discussion.

In subclause 22.9.2.2 [template.bitset.const], update the constructors as follows:

```
constexpr bitset(unsigned long long val) noexcept;  
constexpr bitset(integer-least-int val) noexcept;
```

Effects: Initializes the first M bit positions to the corresponding bit values in val. M is the smaller of N and the ~~number of bits in the value representation~~ width ([basic.types.general]) of ~~unsigned long long~~ integer-least-int. If M < N, the remaining bit positions are initialized to ~~zero~~ one if val is negative, otherwise to zero.

In subclause 22.9.2.3 [bitset.members], make the following changes:

```
constexpr unsigned long to_ulong() const;
```

~~*Returns:* x.~~

~~*Throws:* overflow_error if the integral value x corresponding to the bits in *this cannot be represented as type unsigned long.~~

```
constexpr unsigned long long to_ullong() const;  
template<class T>  
constexpr T to() const;
```

Constraints: T is an unsigned integer type ([basic.fundamental]).

Returns: x.

Throws: overflow_error if the integral value x corresponding to the bits in *this cannot be represented as ~~type unsigned long long~~ the return type of this function.

§ 9.5. Numeric conversions

Update subclause 23.4.2 [string.syn] as follows:

```
string to_string(int val);  
string to_string(unsigned val);  
string to_string(long val);  
string to_string(unsigned long val);  
string to_string(long long val);  
string to_string(unsigned long long val);  
string to_string(int_least128_t);  
string to_string(integer-least-int val);  
string to_string(float val);  
string to_string(double val);  
string to_string(long double val);  
[...]  
wstring to_wstring(int val);  
wstring to_wstring(unsigned val);  
wstring to_wstring(long val);  
wstring to_wstring(unsigned long val);  
wstring to_wstring(long long val);  
wstring to_wstring(unsigned long long val);  
wstring to_wstring(integer-least-int val);  
wstring to_wstring(float val);  
wstring to_wstring(double val);  
wstring to_wstring(long double val);
```

In subclause 23.4.2 [string.syn], add a paragraph:

For each function with a parameter of type *integer-least-int*, the implementation provides an overload for each cv-unqualified integer type ([basic.fundamental]) whose conversion rank is that of *int* or greater, where *integer-least-int* in the function signature is replaced with that integer type.

In subclause 23.4.5 [string.conversions], update `to_string`:

```
string to_string(int val);  
string to_string(unsigned val);  
string to_string(long val);  
string to_string(unsigned long val);  
string to_string(long long val);  
string to_string(unsigned long long val);  
string to_string(integer-least-int val);  
string to_string(float val);  
string to_string(double val);  
string to_string(long double val);
```

Returns: `format("{", val)`.

In subclause 23.4.5 [string.conversions], update `to_wstring`:

```
wstring to_string(int val);  
wstring to_wstring(unsigned val);  
wstring to_wstring(long val);  
wstring to_wstring(unsigned long val);  
wstring to_wstring(long long val);  
wstring to_wstring(unsigned long long val);  
wstring to_wstring(integer-least-int val);  
wstring to_wstring(float val);  
wstring to_wstring(double val);  
wstring to_wstring(long double val);
```

Returns: `format(L"{", val)`.

§ 9.6. Iota view

In subclause 26.6.4.2 [ranges.iota.view], update paragraph 1 as follows:

Let *IOTA-DIFF-T(W)* be defined as follows:

- If *W* is not an integral type, or if it is an integral type and `sizeof(iter_difference_t<W>)` is greater than `sizeof(W)`, then *IOTA-DIFF-T(W)* denotes `iter_difference_t<W>`.
- Otherwise, *IOTA-DIFF-T(W)* is a signed standard integer type of width greater than the width of *W* if such a type exists.
- Otherwise, *IOTA-DIFF-T(W)* is an unspecified signed-integer-like type ([iterator.concept.winc]) of width not less than the width of *W*.
[Note 1: It is unspecified whether this type satisfies `weakly_incrementable`. — end note]

NOTE: This change resolves the potential ABI break explained in [§ 5.2 std::ranges::iota_view ABI issue](#). This change purely increases implementor freedom. An extended integer type still models signed-integer-like, so GCC's existing implementation using `__int128` remains valid. However, a wider extended integer type is no longer the mandatory difference type (if it exists) as per the second bullet.

§ 9.7. Absolute values

In subclause 28.7.1 [cmath.syn], update the synopsis as follows:

```
// [c.math.abs], absolute values
constexpr int abs(int j); // fr
constexpr long int abs(long int j); // fr
constexpr long long int abs(long long int j); // fr
constexpr signed-integer-least-int abs(signed-integer-least-int j); //
constexpr floating-point-type abs(floating-point-type j); //
```

In subclause 28.7.1 [cmath.syn], add a paragraph after paragraph 2:

For each function with a parameter of type *signed-integer-least-int*, the implementation provides an overload for each cv-unqualified signed integer type ([basic.fundamental]) whose conversion rank is that of `int` or greater, where all uses of *signed-integer-least-int* in the function signature are replaced with that signed integer type.

In subclause 28.7.2 [c.math.abs], make the following changes:

```
constexpr int abs(int j);  
constexpr long int abs(long int j);  
constexpr long long int abs(long long int j);  
constexpr signed-integer-least-int abs(signed-integer-least-int j);
```

~~Effects: These functions have the semantics specified in the C standard library for the functions `abs`, `labs`, and `llabs`, respectively.~~ Returns: $j \geq 0 ? j : -j$;

NOTE: $j \geq 0 ? j : -j$; matches the semantics of the C functions exactly, even in undefined cases like `abs(INT_MAX)`.

NOTE: The floating-point overload set is intentionally not re-defined to return $j \geq 0 ? j : -j$. This expression is not equivalent to clearing the sign bit.

§ 9.8. Header `<stdintypes>`

In subclause 31.13.2 [stdintypes.syn], update paragraph 1 as follows:

The contents and meaning of the header `<stdintypes>` are the same as the ~~C standard library~~ C++ header `<inttypes.h>`, with the following changes:

NOTE: Unlike the C standard library header, the C++ *header* has the changes described in [§ 9.3 Header `<inttypes.h>`](#) applied.

§ 9.9. Atomic operations

In subclause 33.5.2 [atomics.syn], update the synopsis as follows:

```
// all freestanding
namespace std {
    [...]

    using atomic_int8_t      = atomic<int8_t>;           // freestanding
    using atomic_uint8_t     = atomic<uint8_t>;          // freestanding
    using atomic_int16_t     = atomic<int16_t>;          // freestanding
    using atomic_uint16_t    = atomic<uint16_t>;         // freestanding
    using atomic_int32_t     = atomic<int32_t>;          // freestanding
    using atomic_uint32_t    = atomic<uint32_t>;         // freestanding
    using atomic_int64_t     = atomic<int64_t>;          // freestanding
    using atomic_uint64_t    = atomic<uint64_t>;         // freestanding
    using atomic_int128_t    = atomic<int128_t>;         // freestanding
    using atomic_uint128_t   = atomic<uint128_t>;        // freestanding

    using atomic_int_least8_t = atomic<int_least8_t>;    // freestanding
    using atomic_uint_least8_t = atomic<uint_least8_t>;  // freestanding
    using atomic_int_least16_t = atomic<int_least16_t>;  // freestanding
    using atomic_uint_least16_t = atomic<uint_least16_t>; // freestanding
    using atomic_int_least32_t = atomic<int_least32_t>;  // freestanding
    using atomic_uint_least32_t = atomic<uint_least32_t>; // freestanding
    using atomic_int_least64_t = atomic<int_least64_t>;  // freestanding
    using atomic_uint_least64_t = atomic<uint_least64_t>; // freestanding
    using atomic_int_least128_t = atomic<int_least128_t>; // freestanding
    using atomic_uint_least128_t = atomic<uint_least128_t>; // freestanding

    using atomic_int_fast8_t  = atomic<int_fast8_t>;    // freestanding
    using atomic_uint_fast8_t = atomic<uint_fast8_t>;    // freestanding
    using atomic_int_fast16_t = atomic<int_fast16_t>;    // freestanding
    using atomic_uint_fast16_t = atomic<uint_fast16_t>;  // freestanding
    using atomic_int_fast32_t = atomic<int_fast32_t>;    // freestanding
    using atomic_uint_fast32_t = atomic<uint_fast32_t>;  // freestanding
    using atomic_int_fast64_t = atomic<int_fast64_t>;    // freestanding
    using atomic_uint_fast64_t = atomic<uint_fast64_t>;  // freestanding
    using atomic_int_fast128_t = atomic<int_fast128_t>;  // freestanding
    using atomic_uint_fast128_t = atomic<uint_fast128_t>; // freestanding
```

```
[...]  
}
```

§ 10. Acknowledgements

I thank Jonathan Wakely and other participants in the std-proposals mailing list whose feedback has helped me improve the quality of this proposal substantially.

I also thank Lénárd Szolnoki for contributing the example in [§ 2.1 Lifting library restrictions](#).

NOTE: See [\[std-proposals\]](#) for discussion of this proposal.

§ References

§ Normative References

[CxxDraft]

VA. *C++ Standard Draft*. URL:

<https://github.com/cplusplus/draft/commit/8238252bcec14f76e97133db32721beaec5c749b>

§ Informative References

[BitPermutations]

Jan Schultke et al.. *C++26 Bit Permutations*. URL: <https://github.com/Eisenwave/cxx26-bit-permutations>

[Bloomberg]

Bloomberg Finance L.P. et al.. *BDE Libraries*. URL: <https://github.com/bloomberg/bde>

[BoostMultiPrecision]

Boost Org. *Boost Multiprecision Library*. URL: <https://github.com/boostorg/multiprecision>

[ClickHouse]

ClickHouse et al.. *ClickHouse*. URL: <https://github.com/ClickHouse/ClickHouse>

[DEShawResearch]

John Salmon et al.. *Random123: a Library of Counter-Based Random Number Generators*. URL: <https://github.com/DEShawResearch/random123>

[Dragonbox]

Junekey Jeon. *Dragonbox: A New Floating-Point Binary-to-Decimal Conversion Algorithm*.

URL: https://github.com/jk-jeon/dragonbox/blob/master/other_files/Dragonbox.pdf

[FAST_FLOAT]

Daniel Lemire et al.. *fast_float number parsing library: 4x faster than strtod*. URL:

https://github.com/fastfloat/fast_float

[LIBDIVIDE]

Kim Walisch et al.. *libdivide*. URL: <https://github.com/ridiculousfish/libdivide>

[Marsaglia]

George Marsaglia. *Xorshift RNGs*. URL: <https://www.jstatsoft.org/article/download/v008i14/916>

[MSDN]

Colen Garoutte-Carson et al.. *Support for 128-bit integer type*. URL:

<https://developercommunity.microsoft.com/t/Support-for-128-bit-integer-type/879048>

[N2341]

ISO/IEC. *Floating-point extensions for C - Decimal floating-point arithmetic*. URL: <https://open-std.org/JTC1/SC22/WG14/www/docs/n2341.pdf>

[N2680]

Robert C. Seacord. *Specific width length modifier*. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2680.pdf>

[N2709]

Anthony Williams. *Packaging Tasks for Asynchronous Execution*. 28 July 2008. URL:

<https://wg21.link/n2709>

[N2763]

Aaron Ballman; et al. *Adding a Fundamental Type for N-bit integers*. URL: <https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf>

[N3047]

ISO. *N3047 working draft — August 4, 2022 ISO/IEC 9899:2023 (E)*. URL: <https://www.iso-9899.info/n3047.html>

[NVIDIA]

Conor Hoekstra; Kuhu Shukla; Mark Harris. *Implementing High-Precision Decimal Arithmetic with CUDA int128*. URL: <https://developer.nvidia.com/blog/implementing-high-precision-decimal-arithmetic-with-cuda-int128/>

[P0543R3]

Jens Maurer. *Saturation arithmetic*. 19 July 2023. URL: <https://wg21.link/p0543r3>

[P1522R1]

Eric Niebler. *Iterator Difference Type and Integer Overflow*. 28 July 2019. URL:

<https://wg21.link/p1522r1>

[P1673R13]

Mark Hoemmen, Daisy Hollman, Christian Trott, Daniel Sunderland, Nevin Liber, Alicia Klinvex, Li-Ta Lo, Damien Lebrun-Grandie, Graham Lopez, Peter Caday, Sarah Knepper, Piotr Luszczek, Timothy Costa. *A free function linear algebra interface based on the BLAS*. 18 December 2023. URL: <https://wg21.link/p1673r13>

[P2075R3]

Ilya Burylov, Ruslan Arutyunyan; Andrey Nikolaev; Alina Elizarova; Pavel Dyakov; John Salmon. *Philox as an extension of the C++ RNG engines*. 13 October 2023. URL: <https://wg21.link/p2075r3>

[P3018R0]

Andreas Weis. *Low-Level Integer Arithmetic*. 15 October 2023. URL: <https://wg21.link/p3018r0>

[PX0]

PikaCat. *Px0*. URL: <https://github.com/official-pikafish/px0>

[RISC-V]

VA. *The RISC-V Instruction Set Manual - Volume I: Unprivileged ISA*. URL: https://drive.google.com/file/d/1s0lZxUZaa7eV_O0_WsZzaurFLlww7ou5/view

[SEC]

U.S. Securities and Exchange Commission. *Division of Market Regulation: Responses to Frequently Asked Questions Concerning Rule 612 (Minimum Pricing Increment) of Regulation NMS*. URL: <https://www.sec.gov/divisions/marketreg/subpenny612faq.htm>

[STD-PROPOSALS]

VA. *[std-proposals] 128-bit integers*. URL: <https://lists.isocpp.org/std-proposals/2024/02/8972.php>

[Stockfish]

Marco Costalba et al.. *Stockfish*. URL: <https://github.com/official-stockfish/Stockfish>

[TigerBeetle]

Rafael Batiati. *64-Bit Bank Balances ‘Ought to be Enough for Anybody’?*. URL: <https://tigerbeetle.com/blog/2023-09-19-64-bit-bank-balances-ought-to-be-enough-for-anybody/>

[Wikipedia]

VA. *Quadruple-precision floating-point format - Hardware support*. URL: https://en.wikipedia.org/wiki/Quadruple-precision_floating-point_format#Hardware_support